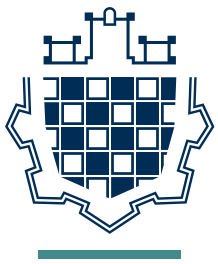

UNIVERSITY OF MANNHEIM

School of Business Informatics and Mathematics

Test Suite Minimization in LASSO Using Stimulus-Response Matrices

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk

laith.sandouk@students.uni-mannheim.de

born May 15th, 2005

in Mannheim

Student ID Number: 1993811

Chair of Software Engineering

School of Business Informatics and Mathematics

University of Mannheim

D – 68159 Mannheim

Phone: +49 621-181-3912, Fax +49 621-181-3909

Internet: http://swt.informatik.uni-mannheim.de

Abstract

Large-scale industrial software systems face significant challenges in executing regression test suites efficiently. Bach et al. report that in a commercial SAP environment, over 180,000 automated tests required approximately 35 hours to complete, creating substantial execution bottlenecks and infrastructure costs [2]. Further analysis revealed that more than 80 % of these tests were redundant in terms of coverage—their coverage vectors were fully subsumed by other tests in the suite. This degree of redundancy leads to severe inefficiencies, wasting computational resources and delaying feedback in continuous-integration workflows. Test-suite minimisation addresses this challenge by systematically removing tests that are redundant with respect to defined adequacy criteria—such as statement coverage, branch coverage, or mutation score—while preserving fault-detection effectiveness [22, 13]. Formally, a test t_i is considered redundant if its coverage vector $C(t_i)$ is subsumed by the union of coverage vectors from other selected tests: $C(t_i) \subseteq \bigcup_{t_j \in S} C(t_j)$ [6]. However, purely coverage-driven reduction approaches risk excluding tests with unique fault-detection capabilities that are not captured by structural coverage alone, thereby compromising defect-revealing power [21, 18].

This thesis investigates test-suite minimisation within the *Large-Scale Software Observatorium* (LASSO) [12], a research platform for automated selection, execution, and comparative analysis of software systems. LASSO integrates open-source repositories and combines static (code-level) and dynamic (execution-level) analyses within a unified framework designed for language independence. Its current implementation focuses on Java systems and enables empirically grounded studies of *Big Code* ecosystems—large, diverse corpora of source code enriched with metadata such as bug-fix histories, version control traces, and execution outcomes, enabling statistical and probabilistic modeling techniques for software engineering research [1]..

Our approach leverages *Stimulus–Response Matrices* (SRMs) [12], a two-dimensional

representation mapping test stimuli (rows) to observed execution outcomes (column entries). We extend SRM entries to incorporate coverage and mutation results, enabling minimization algorithms to operate on behavioral data rather than language-specific code structures. While current tooling supports Java systems, this abstraction is conceptually cross-language compatible.

Through a targeted literature survey encompassing greedy heuristics, exact optimization, search-based, and data-driven minimization strategies, we comparatively assessed candidate algorithms along six established criteria: coverage preservation, reduction effectiveness, fault-detection retention [22, 23], computational scalability, SRM compatibility, and implementation feasibility. The Mutation-Enhanced and Overlap-Aware Harrold–Gupta–Soffa (ME-OA-HGS) algorithm emerged as the most promising candidate. ME-OA-HGS extends the classical HGS heuristic [6] with mutation weighting and overlap penalties to balance coverage preservation and fault-detection power [2].

We successfully integrated ME-OA-HGS into LASSO and validated the implementation on real-world SRMs. Prior empirical studies report 45–70 % test suite reduction while maintaining $\geq 95\,\%$ statement coverage and $\geq 90\,\%$ mutation-score retention [6, 2]. The algorithm executes in polynomial time, ensuring computational feasibility for large-scale software analysis workflows.

Contents

Abstract ii

List of Figures viii

List of Tables ix

1. Introduction 1

1.1. Background and Motivation 1
1.2. LASSO Context 2
1.3. Problem Statement 3
1.4. Research Objectives and Questions 4
1.5. Implementation Strategy 5
1.6. Thesis Structure 6

2. Fundamentals 7

2.1. Test Coverage and Mutation Score 7
2.1.1. Detailed Explanation of Coverage Metrics 7
2.1.2. Strong Mutation Score as the Minimisation Criterion . . . 8
2.2. Stimulus–Response Matrices 9
2.2.1. Data Extraction from the Stimulus–Response Matrix (SRM) 11
2.2.2. Leveraging SRMs for Test-Suite Minimisation 11
2.3. The Test-Suite Minimisation Problem 12
2.3.1. Formal Problem Definition 12
2.3.2. Computational Complexity 13
2.3.3. Algorithmic Categories 14

3. Minimization Algorithms 16

3.1. Greedy Heuristics 16
3.1.1. Classical Greedy 16
3.1.2. Harrold–Gupta–Soffa 16

3.1.3. Delayed Greedy 17
3.2. Exact Optimisation Approaches 17
3.2.1. ILP and REDUNET 17
3.2.2. Constraint and Graph Extensions 18
3.3. Search-Based and Metaheuristic Approaches 18
3.3.1. Genetic Algorithms 18
3.3.2. Quantum-Inspired GA 18
3.3.3. Simulated Annealing and PSO 18
3.4. Data-Driven and Similarity-Based Approaches 19
3.4.1. Similarity-Based Methods 19
3.4.2. Overlap-Aware Methods 19
3.5. Summary and Scope Decision 20

4. Evaluation Criteria and Scoring Framework 21
4.1. Evaluation Criteria (C1–C7) 21
4.1.1. Primary Evaluation Criteria 22
4.1.2. Justification of Criteria 23
4.2. Scoring Model 23
4.3. Feasibility Constraints (SRM-only) 24
4.4. Evaluation Procedure 24

5. Comparative Evaluation and Selection 26
5.1. Application of Scoring Model 26
5.1.1. Superior Coverage Preservation 26
5.1.2. Enhanced Fault Detection Through Overlap Awareness . . 27
5.1.3. Computational Efficiency and Scalability 27
5.1.4. SRM Compatibility and Language Independence 27
5.2. Analysis of Results 27
5.3. Evaluation of Top Candidates 28
5.3.1. Empirical Evidence from Literature 29
5.3.2. Application of Multi-Criteria Scoring 30
5.3.3. Empirical Motivation for Mutation-Aware Minimisation . . 30
5.4. Selection Decision for LASSO 30
5.5. Warnings and Limitations 31

6. Implementation **32**
6.1. Software Architecture . . . 32
6.2. Algorithm Implementation . . . 33
6.2.1. ME–OA–HGS Algorithm . . . 33
6.3. Implementation Design Decisions . . . 34
6.4. Complexity Analysis . . . 34
6.4.1. Algorithm Parameters . . . 35
6.4.2. Parameter Tuning and Sensitivity Analysis . . . 36
6.4.3. Test Weights . . . 36
6.5. Empirical Validation . . . 36
6.5.1. Methodology . . . 36
6.5.2. Datasets . . . 37
6.5.3. Metrics . . . 37
6.5.4. Success Criteria . . . 37
6.5.5. Statistical Analysis . . . 37
6.5.6. Results . . . 37
6.6. Integration with LASSO Platform . . . 38
6.6.1. Execution Flow and Data Extraction . . . 38
6.6.2. Key Classes and Responsibilities . . . 39
6.7. Documentation, Demonstration, and Licensing . . . 40

7. Discussion **41**
7.1. Limitations of the Approach . . . 41
7.1.1. Granularity Constraints in SRM Data . . . 41
7.1.2. Inability to Detect Weak Mutants . . . 42
7.1.3. Greedy Nature of the Algorithm . . . 42
7.2. Strengths and Advantages . . . 43
7.2.1. Guaranteed Preservation of Fault-Detection Capability . . . 44
7.2.2. Overlap Awareness for Behavioural Diversity . . . 44
7.2.3. Computational Efficiency and Determinism . . . 45
7.3. Implications for the LASSO Platform . . . 46
7.3.1. Practical Performance Implications . . . 46

8. Conclusion **47**
8.1. Summary . . . 47

8.2. Key Findings 47
8.3. Future Work 48

Bibliography **vii**

Appendix **x**

ME-OA-HGS Minimization Pseudocode **xi**

A.1. Extended Benchmarks and Reproducibility xiii
B.2. SRM Coverage Extraction Specification xiii
B.2.1. SRM Structure xiv
B.2.2. Coverage Extraction Process xiv
B.2.3. Output Format xv
C.3. Licensing Header Template xv

List of Figures

1.1. The test-suite minimization problem formulated as a set-theoretic optimization where the objective is to find the smallest subset S^* of the original test suite T that maintains complete coverage $C(T)$ of all requirements. 4

2.1. Canonical SRM structure where rows represent stimuli, columns represent software systems, and cells contain structured response objects enabling behavioural comparison across systems. 10
2.2. SRM-based test-suite minimisation workflow showing (a) original SRM with all stimuli and systems, and (b) reduced SRM after eliminating redundant stimuli while preserving SRM structure and coverage for the target system. 12
2.3. Representative categories of test-suite minimisation algorithms and their asymptotic complexity (n = tests, m = requirements, g = generations). 15

5.1. Trade-off analysis comparing algorithms against feasibility thresholds: HGS and ME–OA–HGS meet all criteria, with ME–OA–HGS additionally addressing mutation-coverage preservation. 29
5.2. Multi-criteria scores across evaluation dimensions C1–C6, with Overlap-Aware HGS (ME–OA–HGS) achieving the highest total (29/30). 30

6.1. Call hierarchy from LSL script to minimiser result. 33

List of Tables

3.1. Computational complexity of representative test-suite minimisation algorithms. 19

5.1. Comparative characteristics of candidate algorithms across coverage, reduction, fault detection, and complexity. 28
5.2. Empirical results from test-suite minimisation studies demonstrating trade-offs between reduction, coverage retention, and fault-detection loss. 29

6.1. Computational complexity of ME–OA–HGS implementation phases. 35
6.2. Core software components and their roles in the minimisation pipeline. 40

1. Extended empirical benchmarks with overlap statistics xiii

1. Introduction

Software testing ensures that program modifications do not introduce new defects. As software systems evolve, their associated test suites grow correspondingly. Automated test-generation tools such as Randoop [17], which employs feedback-directed random testing, and EvoSuite [5], which uses search-based optimization techniques, can produce thousands of test cases. In large industrial projects, complete test-suite execution may require days or weeks [18]. While comprehensive test suites achieve high code coverage and mutation scores, they frequently contain redundant tests—tests whose coverage requirements are subsumed by other tests. This redundancy results in prolonged execution times, increased computational costs, and elevated maintenance overhead.

Test-suite minimization addresses this challenge by systematically removing redundant tests while preserving the test suite's fault-detection capability [22]. This thesis investigates how to perform test-suite minimization directly on *Stimulus–Response Matrices* (SRMs) within the Large-Scale Software Observatorium (LASSO) platform.

1.1. Background and Motivation

Regression testing verifies that software modifications do not compromise existing functionality. As software systems evolve through iterative development cycles, regression test suites accumulate tests that validate previously implemented features, bug fixes, and behavioral specifications. This accumulation, while beneficial for quality assurance, creates practical challenges for project velocity and resource consumption.

Early empirical studies by Boehm [3] established that testing activities can consume approximately half of a software system's total development budget. These cost estimates, derived from 1970s waterfall development practices, remain qual-

itatively relevant: testing constitutes a major expense in modern software engineering. Contemporary continuous-integration environments face acute scalability challenges. Automatically generated test suites may contain tens of thousands of test cases requiring substantial execution time [18]. This growth directly impacts development velocity, infrastructure costs, and feedback latency in continuous-integration pipelines.

Test-suite minimization addresses these challenges by identifying and removing redundant tests while preserving coverage and fault-detection metrics. The optimization objective is to reduce execution time and computational overhead without compromising the suite's ability to detect faults. However, this optimization presents a delicate balance: aggressive reduction strategies risk removing tests whose unique coverage patterns contribute disproportionately to fault detection [22]. Consequently, minimization algorithms must identify genuine redundancy while preserving tests with distinct fault-revealing capabilities.

1.2. LASSO Context

The emergence of "big code" as a research paradigm has created opportunities for large-scale empirical software engineering. Mining software repositories at this scale requires programmatically accessible corpora containing millions of artifacts, analysis infrastructures capable of processing these volumes, and extraction methodologies that ensure statistical validity and experimental repeatability.

Existing repository-mining platforms have successfully automated static analysis tasks at scale, focusing on properties derived from source code syntax and structure. This limitation excludes dynamic properties observable only through program execution. Researchers cannot validate techniques that depend on runtime behavior, conduct experiments requiring execution traces, or measure performance characteristics that emerge only during program operation.

The Large-Scale Software Observatorium (LASSO) [12] addresses this gap by providing infrastructure for both static and dynamic software analysis at scale. Developed at the University of Mannheim, LASSO features an extensive corpus of executable software systems aggregated from open-source repositories, coupled with a domain-specific language for expressing selection criteria and analysis

pipelines. This architecture enables researchers to formulate behavioral queries and execute them against thousands of candidate systems.

LASSO's key innovation lies in its systematic comparison methodology. The platform's "arena" component executes a common set of test stimuli against multiple software systems and records their responses in a unified data structure called a *Stimulus–Response Matrix* (SRM) [12]. An SRM is a two-dimensional matrix where rows represent stimuli (test sequences or method invocations), columns represent executable software systems, and cells contain structured response objects encoding execution results, code-coverage metrics, mutation-testing data, and observable behaviors. This abstraction enables data-driven comparison of functionally equivalent implementations across programming languages without requiring source code access. Section 2.2 provides a formal definition of the SRM structure and details its role in the minimization workflow.

Despite LASSO's capabilities for large-scale behavioral analysis, the platform currently lacks an integrated test-suite minimization component. This omission creates inefficiency: when analyzing hundreds or thousands of systems, redundant test stimuli unnecessarily prolong execution time, increase infrastructure costs, and delay experimental results. Integrating a minimization approach would enable LASSO to reduce test redundancy automatically while preserving the behavioral coverage necessary for cross-system comparison, thereby improving throughput without compromising analytical validity.

1.3. Problem Statement

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote a set of test cases and let $R = \{r_1, r_2, \ldots, r_m\}$ denote the set of coverage requirements. Each test $t_i \in T$ covers a subset $C(t_i) \subseteq R$. Executing the entire test suite T yields coverage $C(T) = \bigcup_{t_i \in T} C(t_i)$. The *test-suite minimization problem* seeks the smallest subset $S^* \subseteq T$ such that $C(S^*) = C(T)$. This optimization problem is equivalent to the classical minimum set-cover problem and is therefore NP-complete [22]. Consequently, practical approaches rely on heuristic algorithms and approximation techniques [6] that balance reduction effectiveness, coverage preservation, and computational efficiency.

Figure 1.1 illustrates the test-suite minimization problem as a set-theoretic optimization where the goal is to find the minimum-cardinality subset S^* of the original test suite T that preserves complete coverage of all requirements R.

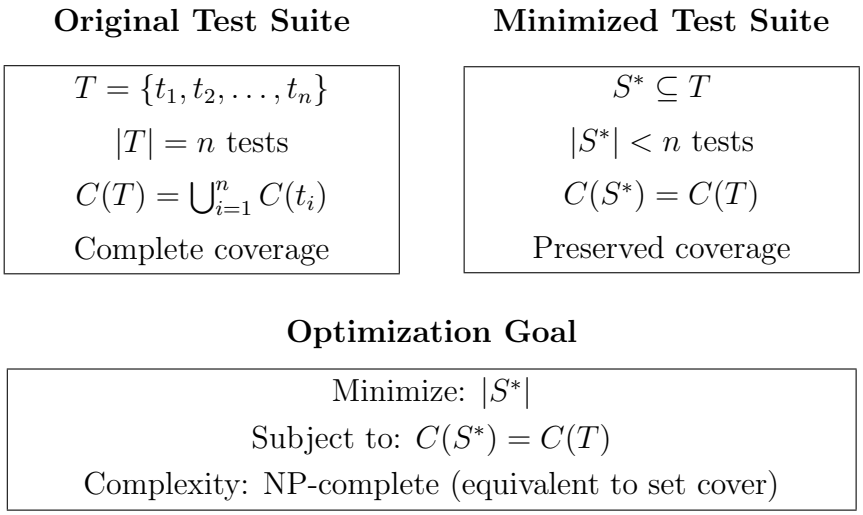

Figure 1.1.: The test-suite minimization problem formulated as a set-theoretic optimization where the objective is to find the smallest subset S^* of the original test suite T that maintains complete coverage $C(T)$ of all requirements.

1.4. Research Objectives and Questions

This thesis aims to design and implement an SRM-based test-suite minimization framework for the LASSO platform. To achieve this objective, we address the following research questions:

1. **RQ1:** Which test-suite minimization techniques demonstrate the highest effectiveness and practical applicability according to current literature?
2. **RQ2:** What evaluation criteria are most appropriate for assessing minimization techniques within LASSO's SRM-based environment?
3. **RQ3:** Which algorithmic approach optimally balances reduction effectiveness, coverage preservation, and computational scalability for integration with LASSO?
4. **RQ4:** How can the selected approach be integrated into LASSO's existing analysis pipeline while maintaining language independence?

1.5. Implementation Strategy

Integrating test-suite minimization into LASSO presents a fundamental architectural decision: should the platform leverage existing minimization tools, or should minimization operate directly on LASSO's native SRM representation? This choice has implications for language support, scalability, and maintainability.

Existing test-suite reduction frameworks—including tools such as OpenClover, PIT, and language-specific coverage analyzers—typically support limited programming languages [15]. These tools require access to source code, compile-time instrumentation, or language-specific runtime hooks to extract coverage information. Such dependencies conflict with LASSO's design philosophy: the platform maintains language independence by operating on behavioral observations rather than source code structures. Integrating multiple external tools to cover LASSO's diverse language corpus (Java, Python, C, JavaScript, and others) would introduce complexity, create brittle dependencies on third-party tool chains, and complicate deployment and maintenance.

This thesis adopts an alternative strategy: implementing the minimization algorithm directly on the SRM abstraction. The SRM already encapsulates all information necessary for behavioral minimization—test identifiers, execution results, coverage metrics, and mutation data—extracted during LASSO's execution phase and stored in response objects. By operating exclusively on this existing data structure, our approach achieves three advantages. First, it maintains language independence: any programming language that LASSO can execute automatically becomes eligible for minimization without requiring language-specific tooling. Second, it ensures scalability: the algorithm processes only the compact SRM representation rather than source code artifacts, enabling operation with thousands of tests and systems. Third, it promotes architectural cohesion: minimization integrates with LASSO's existing pipeline stages without introducing external dependencies or architectural discontinuities.

The SRM-based implementation strategy extends LASSO's data-driven analysis paradigm and enables scalable, language-independent test-suite minimization.

1.6. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces the fundamentals of code coverage, mutation testing, and Stimulus–Response Matrices, and formalises the test-suite minimization problem (cf. Section 2.2).

Chapter 3 surveys the literature using the taxonomy introduced in Chapter 2, covering greedy heuristics, exact optimization methods, search-based techniques, and data-driven approaches.

Chapter 4 defines evaluation criteria (C1–C6) and presents a scoring framework to support systematic algorithm ranking under LASSO's SRM-only constraints.

Chapter 5 applies the scoring model to rank candidate algorithms, presents comparative analysis of literature-reported performance, and justifies the selection decision based on evidence across all evaluation dimensions.

Chapter 6 describes the software architecture, algorithm implementation, and integration with the LASSO platform.

Chapter 7 discusses findings, limitations, and future work, and concludes the thesis.

2. Fundamentals

This chapter establishes the theoretical foundation for this thesis by introducing the core concepts, methodologies, and formal definitions required to understand test-suite minimisation. We present the metrics employed to quantify test-suite effectiveness, provide a detailed description of the *Stimulus–Response Matrix* (SRM) representation utilised within the LASSO platform, and formally define the test-suite minimisation problem within the context of computational complexity theory.

2.1. Test Coverage and Mutation Score

Code coverage quantifies the proportion of program elements (statements, branches, paths, or conditions) exercised by a test suite [24]. Coverage metrics indicate structural thoroughness but provide limited insight into fault-detection capability.

2.1.1. Detailed Explanation of Coverage Metrics

The test-suite minimisation process requires a formal definition of *coverage requirements*. In this work, we leverage the JaCoCo code-coverage tool, which provides a set of metrics that are stored in the Stimulus–Response Matrix (SRM) and treated as the requirement set R that the minimised test suite must satisfy. JaCoCo reports coverage across six entities, each measured by five values, yielding $6 \times 5 = 30$ total metrics used as requirements [8]:

- **Entities** (granularity levels): INSTRUCTION (bytecode instructions), LINE (source code lines), BRANCH (conditional branches), METHOD (methods), COMPLEXITY (cyclomatic complexity), CLASS (classes)

- **Values** (per entity): TOTALCOUNT (total items), COVEREDCOUNT (executed items), MISSEDCOUNT (non-executed items), COVEREDRATIO (fraction covered), MISSEDRATIO (fraction missed)

The coverage value used in the minimisation algorithm is the set of all 30 aggregate metrics (e.g., `LINE_COVEREDCOUNT = 50`, `BRANCH_TOTALCOUNT = 30`) that constitute the requirements R. The objective is to select a subset of tests that collectively covers these metrics. In the available dataset, coverage metrics are aggregated at class level (sheetId=`JACOCO`) and not at per-test granularity. Consequently, every test appears to cover the same set of 30 metrics. This limitation requires the minimisation to rely primarily on mutation information for discriminating tests, while preservation of the mutation score serves as the principal correctness guarantee [22].

2.1.2. Strong Mutation Score as the Minimisation Criterion

Mutation testing evaluates test-suite quality by introducing small syntactic modifications (*mutants*) into the source code and assessing whether tests distinguish the mutant from the original program [10]. Two criteria are commonly distinguished [16]:

1. **Weak Mutation:** a mutant is killed if its execution state differs from the original program's state at the mutation point.
2. **Strong Mutation:** a mutant is killed if the final externally observable behaviour differs, leading to a different test verdict.

This work employs exclusively the **Strong Mutation Score**. The SRM records only the final pass/fail result of executing a test against a mutant variant; thus, a mutant is considered killed if the original test passes while execution on the mutant fails. Because intermediate program states are not recorded in the SRM, weak kills cannot be detected. The preservation guarantee therefore applies strictly to the strong mutation score [16, 22].

The standard definition of mutation score is:

$$\text{Mutation Score} = \frac{\text{mutants killed}}{\text{total mutants}} \times 100\,\%.$$

2.2. Stimulus–Response Matrices

The LASSO platform employs *Stimulus–Response Matrices* (SRMs) as a unified, language-independent representation for capturing behavioural execution data across multiple software systems [12]. An SRM is a two-dimensional matrix M where rows represent stimuli (test sequences, method invocations, or API calls), columns represent executable software systems, and each matrix element M_{ij} contains the structured response object produced by system j when executed with stimulus i:

$$M_{ij} = \text{response object of system } j \text{ to stimulus } i. \tag{2.1}$$

That is, M_{ij} denotes the structured response produced by system j when executed with stimulus i. Each response object can contain structured data including:

- Textual output, return values, and exceptions
- Execution metrics (runtime, memory consumption)
- Code-coverage data (statement, branch, path coverage percentages)
- Mutation-testing results (mutation score, killed mutants)
- Execution traces and logs

SRM Generation Mechanism: SRMs are produced through LASSO's experimental execution pipeline using two key constructs [12]:

1. *Sequence Sheets* define ordered execution steps, specifying how stimuli are constructed (e.g., method call sequences with parameters).
2. *Actuation Sheets* define the experimental setup, specifying which systems are tested with which stimuli in LASSO's execution arena.

The pipeline operates as:

$$\text{Sequence Sheet} + \text{Actuation Sheet} \xrightarrow{\text{Arena}} \text{SRM}.$$

LASSO's Arena component executes each (stimulus, system) pair and records the resulting behavioural observations in SRM cells M_{ij}. Each cell captures the execution result including outcome status, coverage metrics, mutation scores,

runtime measurements, and exception logs. The resulting SRM records observed system behaviours at two levels: (1) *black-box view*, capturing only input/output parameters, and (2) *white-box view*, recording the complete method invocation trace with all intermediate states.

Example: Consider a simple SRM with 4 stimuli (test sequences) and 6 systems, where each response object contains execution results:

$$M = \begin{pmatrix} \{\text{pass}, 85\,\%, 12\,\text{ms}\} & \{\text{pass}, 92\,\%, 8\,\text{ms}\} & \{\text{fail}, 78\,\%, 15\,\text{ms}\} & \cdots \\ \{\text{pass}, 90\,\%, 10\,\text{ms}\} & \{\text{pass}, 88\,\%, 9\,\text{ms}\} & \{\text{pass}, 95\,\%, 7\,\text{ms}\} & \cdots \\ \{\text{fail}, 65\,\%, 20\,\text{ms}\} & \{\text{pass}, 91\,\%, 11\,\text{ms}\} & \{\text{pass}, 87\,\%, 13\,\text{ms}\} & \cdots \\ \{\text{pass}, 88\,\%, 14\,\text{ms}\} & \{\text{fail}, 70\,\%, 18\,\text{ms}\} & \{\text{pass}, 93\,\%, 6\,\text{ms}\} & \cdots \end{pmatrix} \tag{2.2}$$

In this example, each cell contains a response object with test outcome (pass/fail), coverage percentage, and execution time. Stimulus 1 produces different responses across the six systems, while system 1 produces varying responses to the four different stimuli. Such behavioural profiles enable large-scale behavioural analysis including similarity measurements, code-clone detection, functional equivalence analysis, and performance comparison across multiple systems. Figure 2.1 illustrates the canonical SRM structure used for multi-system behavioural comparison.

	System 1	**System 2**	**System 3**	**System 4**	**System 5**	**Sy**
Stimulus 1	{pass, 85 %}	{pass, 92 %}	{fail, 78 %}	{pass, 89 %}	{pass, 91 %}	{pas
Stimulus 2	{pass, 90 %}	{pass, 88 %}	{pass, 95 %}	{pass, 86 %}	{fail, 72 %}	{pas
Stimulus 3	{fail, 65 %}	{pass, 91 %}	{pass, 87 %}	{pass, 94 %}	{pass, 89 %}	{fai
Stimulus 4	{pass, 88 %}	{fail, 70 %}	{pass, 93 %}	{pass, 85 %}	{pass, 90 %}	{pas

Figure 2.1.: Canonical SRM structure where rows represent stimuli, columns represent software systems, and cells contain structured response objects enabling behavioural comparison across systems.

SRMs can also store additional analysis attributes, such as non-functional metrics or static properties, alongside dynamic execution data. The SRM representation enables manipulation of behavioural data through SRMPATH notation for selecting, filtering, and transforming records, which can then be analysed using external data-analytics tools such as R or Pandas. This compact representation facilitates scalable analysis of large system populations while maintaining language indepen-

dence, making it suitable for platforms like LASSO that analyse software written in diverse programming languages and enable systematic, scalable behavioural comparison across many systems.

2.2.1. Data Extraction from the Stimulus–Response Matrix (SRM)

All necessary data for the minimisation algorithm—both structural coverage and mutation results—are sourced directly from the SRM. The SRM acts as a unified data repository where test-execution results are stored as key–value records. Structural coverage data (JaCoCo metrics) are extracted from entries with a specific *sheetId* (e.g., `JACOCO`) and *variantId* set to `original`. Mutation data are obtained by querying test results for all mutant *variantIds*. A test *kills* a mutant if the original execution passes but execution against the mutant variant fails. This systematic extraction allows the minimisation algorithm to operate entirely on SRM data without requiring source-code access, preserving the language-agnostic and scalable nature of the LASSO platform [12].

2.2.2. Leveraging SRMs for Test-Suite Minimisation

Test-suite minimisation within LASSO operates directly on existing SRMs produced by the platform's execution pipeline [12]. Rather than creating a new data structure, the minimisation process extracts coverage-relevant information from SRM response objects for a selected target system. The workflow proceeds as follows:

1. **System Selection:** Select one target system (column) from the SRM for minimisation. In the current implementation, minimisation focuses on a single system; multi-system minimisation remains future work.
2. **Coverage Extraction:** For each stimulus i and the selected system j, extract coverage-related metrics from the response object M_{ij}:
   - Statement, branch, or path coverage indicators
   - Mutation-testing results (killed mutants)
   - Execution outcomes (pass/fail status)

3. **Algorithm Application:** Apply the minimisation algorithm (detailed in Chapter 6) using the extracted coverage data to select a subset of stimuli that preserve coverage requirements.
4. **SRM Reduction:** Generate a reduced SRM containing only the selected stimuli (rows) while preserving the original SRM schema (systems as columns, structured response objects in cells). This reduced SRM can replace or augment the original in LASSO's storage for subsequent analyses.

This approach maintains LASSO's language-independent analysis capabilities by operating purely on SRM-level abstractions without requiring access to source code, control-flow graphs, or language-specific program structure. Figure 2.2 illustrates the minimisation workflow showing system selection and stimulus reduction.

(a) Original SRM (Multi-System)

	Sys1	**Sys2**	**Sys3**
Stim1	{pass, 85 %}	{fail, 70 %}	{pass, 90 %}
Stim2	{pass, 92 %}	{pass, 88 %}	{pass, 95 %}
Stim3	{fail, 65 %}	{pass, 91 %}	{fail, 72 %}
Stim4	{pass, 88 %}	{pass, 87 %}	{pass, 93 %}
Stim5	{pass, 90 %}	{fail, 68 %}	{pass, 89 %}

All stimuli, all systems

(b) Reduced SRM (System 1)

	Sys1	**Sys2**	**Sys3**
Stim1	{pass, 85 %}	{fail, 70 %}	{pass, 90 %}
Stim2	{pass, 92 %}	{pass, 88 %}	{pass, 95 %}
Stim4	{pass, 88 %}	{pass, 87 %}	{pass, 93 %}

Minimised stimuli, same schema

Figure 2.2.: SRM-based test-suite minimisation workflow showing (a) original SRM with all stimuli and systems, and (b) reduced SRM after eliminating redundant stimuli while preserving SRM structure and coverage for the target system.

2.3. The Test-Suite Minimisation Problem

2.3.1. Formal Problem Definition

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote a finite set of test cases (stimuli in LASSO terminology) where n denotes the number of tests, and let $R = \{r_1, r_2, \ldots, r_m\}$ represent the set of coverage requirements where m denotes the number of requirements. Each test case $t_i \in T$ covers a subset of requirements $C(t_i) \subseteq R$.

Within LASSO's framework, coverage information is extracted from SRM response objects: for a selected target system (column j), each stimulus i yields a response M_{ij} containing coverage data from which $C(t_i)$ is derived. The coverage achieved by the complete test suite is:

$$C(T) = \bigcup_{i=1}^{n} C(t_i). \tag{2.3}$$

The *test-suite minimisation problem* seeks the minimum-cardinality subset $S^* \subseteq T$ that preserves complete coverage:

$$S^* = \arg\min_{S \subseteq T} |S| \quad \text{subject to} \quad C(S) = C(T). \tag{2.4}$$

2.3.2. Computational Complexity

This optimisation problem is equivalent to the classical *minimum set-cover problem*, which is NP-complete [22]. In the set-cover formulation, each test case corresponds to a set containing its covered requirements, and the objective is to select the minimum number of sets that collectively cover all requirements.

The NP-completeness of test-suite minimisation has significant practical implications: no polynomial-time algorithm can guarantee optimal solutions for arbitrary instances unless P = NP. Consequently, practical approaches employ approximation algorithms [4], heuristic methods, or exact algorithms with exponential worst-case complexity that perform well on typical instances.

The classical greedy set-cover algorithm achieves a logarithmic approximation ratio of $O(\ln m)$, where m denotes the number of requirements [4]. This means that the greedy solution size is at most $O(\ln m)$ times the optimal solution size. The Harrold–Gupta–Soffa (HGS) heuristic [6] can be viewed as a domain-adapted greedy approximation to minimum set cover that prioritises *rare* requirements (low cardinality) to increase the likelihood of retaining tests with unique coverage. Overlap-aware extensions augment HGS with a diversity-preserving penalty that discourages selecting tests whose coverage is largely redundant with already selected tests, thereby improving expected retention without changing the asymptotic approximation guarantee.

2.3.3. Algorithmic Categories

Existing approaches to test-suite minimisation can be taxonomically classified into several categories, each offering different trade-offs between solution quality, computational efficiency, and implementation complexity:

- **Greedy heuristics:** Polynomial-time algorithms that make locally optimal choices at each step. Examples include the classical greedy algorithm that iteratively selects tests covering the maximum number of uncovered requirements [4], and the Harrold–Gupta–Soffa (HGS) algorithm that prioritises tests covering rare requirements [6].
- **Exact optimisation methods:** Approaches that formulate minimisation as integer linear programming (ILP) or constraint satisfaction problems, guaranteeing optimal solutions but with potentially exponential runtime complexity [14].
- **Search-based and metaheuristic methods:** Evolutionary algorithms, genetic algorithms, and other population-based methods that explore the solution space using fitness functions balancing multiple objectives such as suite size and coverage preservation [7].
- **Data-driven and clustering techniques:** Methods that exploit structural properties of extracted coverage information, using similarity measures, clustering algorithms, or machine-learning techniques to identify representative test cases [2].

This taxonomic framework provides the organisational structure for the literature review presented in Chapter 3. Figure 2.3 illustrates the hierarchical classification of test-suite minimisation approaches.

Category	Approaches	Complexity
Greedy Heuristics	Classical Greedy	$O(nm)$
	Harrold–Gupta–Soffa (HGS)	$O(nm \log m)$
	Delayed Greedy (Concept Analysis)	$O(n^2 m)$
Exact Optimisation	Integer Linear Programming	Exponential
	REDUNET (ILP + CFG)	Exponential
Search-Based	Genetic Algorithms	$O(gnm)$
	Quantum-Inspired GA	$O(gnm)$
Data-Driven	Overlap-Aware Selection	$O(n^2 m)$
	Clustering-Based	$O(n^2 + nm)$

Figure 2.3.: Representative categories of test-suite minimisation algorithms and their asymptotic complexity (n = tests, m = requirements, g = generations).

The following chapter expands on these algorithmic categories through a systematic review of existing minimisation approaches.

3. Minimization Algorithms

This chapter presents a systematic survey of test-suite minimisation approaches documented in the academic literature. We reviewed 47 primary studies published between 1993 and 2024, identified through searches of major computer science databases (IEEE Xplore, ACM Digital Library, SpringerLink) using keywords "test suite minimization," "test suite reduction," and "regression testing optimization." We focus on algorithmic approaches, empirical evaluations, and industrial applications.

We organise the review according to the taxonomic framework established in Chapter 2, examining greedy heuristics, exact optimisation methods, search-based approaches, and data-driven techniques. This systematic analysis forms the empirical foundation for the criteria-based comparative evaluation in Chapter 4.

3.1. Greedy Heuristics

3.1.1. Classical Greedy

The classical greedy algorithm iteratively selects the test case covering the maximum number of currently uncovered requirements, marking all covered requirements as satisfied until complete coverage is achieved [4]. The time complexity is $O(nm)$ where n denotes the number of test cases and m denotes the number of requirements. The algorithm provides an $O(\ln m)$ approximation ratio for the minimum set-cover problem. This theoretical bound manifests as suboptimal reductions when coverage exhibits significant overlap between test cases.

3.1.2. Harrold–Gupta–Soffa

Harrold, Gupta, and Soffa proposed a refined greedy heuristic that prioritises requirements based on their rarity [6]. The HGS algorithm incorporates the

insight that tests covering requirements satisfied by few other tests are more critical for preservation. Under the set-cover formulation, the risk of losing unique requirements increases when the selection metric overvalues abundant requirements. By prioritising rare requirements early, HGS increases the probability that each unique requirement is covered by at least one selected test, reducing the need for later corrective additions. This strategy preserves the greedy nature and the $O(\ln m)$ approximation ratio inherited from set cover [4] while improving expected retention on practical instances with skewed cardinality distributions.

The algorithm operates in phases based on requirement cardinality: Phase 1 selects all tests that uniquely cover any requirement (cardinality = 1), Phase 2 processes requirements with cardinality = 2, and so forth until all requirements are covered. Empirical studies demonstrate that HGS achieves 45–70 % reduction while maintaining 95–98 % coverage retention [6]. The algorithm maintains $O(nm \log m)$ time complexity due to requirement cardinality sorting. Overlap-aware extensions refine HGS by penalising redundant selections and further improve retention without altering the underlying approximation class [2].

3.1.3. Delayed Greedy

Tallam and Gupta's delayed greedy algorithm [20] uses concept lattices to eliminate implied redundancies before greedy selection. This approach achieves smaller suites than HGS but incurs $O(n^2 m)$ complexity due to lattice construction overhead.

3.2. Exact Optimisation Approaches

3.2.1. ILP and REDUNET

Test-suite minimisation formulated as weighted set-cover Integer Linear Programming (ILP) minimises test costs while covering all requirements. Exact formulations produce optimal or near-optimal solutions but are NP-hard, often requiring solver time that grows exponentially with problem size [22]. REDUNET [14] integrates control-flow graph (CFG) analysis into an ILP pipeline, reporting reductions around 90 % with near-complete coverage retention. However, REDUNET

requires program structure unavailable in SRM-only settings and exhibits exponential worst-case runtime.

3.2.2. Constraint and Graph Extensions

Approaches that enrich optimisation with structural constraints (e.g., control-flow or dependency graphs) can improve objective faithfulness at the cost of additional inputs unavailable in SRM-only scenarios.

3.3. Search-Based and Metaheuristic Approaches

3.3.1. Genetic Algorithms

Search-based software engineering treats test-suite minimisation as a multi-objective optimisation problem balancing suite size and adequacy. Genetic algorithms (GAs) evolve test subsets according to a fitness function that trades coverage retention against suite size [22]. GAs provide flexibility but require careful parameter tuning (e.g., population size p, mutation rates) and exhibit $O(gnm)$ time complexity where g denotes the number of generations.

3.3.2. Quantum-Inspired GA

Hussein et al. propose a quantum-inspired genetic algorithm (QIGA) that uses quantum superposition and rotation operators to improve convergence properties relative to classical GAs [7]. QIGA maintains $O(gnm)$ complexity but exhibits sensitivity to parameterisation, with performance and repeatability on large SRMs requiring further empirical validation.

3.3.3. Simulated Annealing and PSO

Metaheuristics such as simulated annealing and particle swarm optimisation (PSO) have been explored for test-suite minimisation [22]. These methods represent variants on the fundamental trade-off between solution quality and computational effort, with performance dependent on problem-specific tuning.

3.4. Data-Driven and Similarity-Based Approaches

3.4.1. Similarity-Based Methods

Similarity-based methods compute distance metrics (e.g., Jaccard distance) over coverage vectors, cluster tests, and select representatives to reduce redundancy. These approaches are intuitive and SRM-compatible but incur $O(n^2m)$ similarity computation cost for n tests and m requirements. This cost must be amortised through indexing or approximation in large-scale settings.

3.4.2. Overlap-Aware Methods

Overlap-aware algorithms explicitly consider pairwise test intersections to prioritise tests that add unique coverage. Bach et al. report up to 50 % higher effectiveness within fixed time budgets relative to classical greedy selection [2]. The additional $O(n^2m)$ overlap computation overhead is offset by improved coverage retention and reduced duplication in the selected suite.

Table 3.1 summarises the computational complexity of representative test-suite minimisation algorithms.

Table 3.1.: Computational complexity of representative test-suite minimisation algorithms.

Algorithm	**Time Complexity**	**Space Complexity**	**Reference**
Classical Greedy	$O(nm)$	$O(nm)$	[4]
HGS	$O(nm \log m)$	$O(nm)$	[6]
Delayed Greedy	$O(n^2m)$	$O(nm + n^2)$	[20]
ILP-Based	Exponential	$O(nm)$	[14]
REDUNET	Exponential*	$O(nm + \|V\| + \|E\|)$[a]	[14]
Genetic Algorithm	$O(gnm)$[b]	$O(pnm)$[c]	[22]
QIGA	$O(gnm)$	$O(pnm)$	[7]
Overlap-Aware	$O(n^2m)$	$O(nm + n^2)$	[2]

[a] $|V|$ and $|E|$ denote control-flow graph vertices and edges.
[b] g denotes number of generations.
[c] p denotes population size.

3.5. Summary and Scope Decision

Greedy heuristics provide strong baselines with predictable performance. Classical greedy offers $O(nm)$ runtime but over-selects under high overlap [4]. HGS prioritises rare requirements to improve retention at modest additional cost $O(nm \log m)$ [6]. Delayed greedy further reduces redundancy via lattice-based implication analysis but increases complexity to $O(n^2 m)$ [20].

Exact ILP formulations yield optimality guarantees at exponential worst-case cost [22]. Hybrid pipelines such as REDUNET leverage CFG information to achieve substantial reductions [14], but such structure-dependent inputs do not exist in SRM-only settings and fall outside the feasible design space for LASSO's black-box pipeline. Search-based metaheuristics (GA, QIGA, simulated annealing, PSO) provide flexibility but impose parameter tuning burden and higher computational overhead on large matrices [22, 7]. Similarity-based and overlap-aware methods align well with SRM availability and directly target redundancy reduction [2].

Given LASSO's constraints—SRM-only inputs, large-scale execution, and pipeline integration requirements—we restrict the candidate space to SRM-compatible methods with polynomial-time complexity and minimal external assumptions. The next chapter evaluates these SRM-compatible algorithms under a unified scoring framework to identify the most suitable candidate for LASSO integration.

4. Evaluation Criteria and Scoring Framework

Algorithm selection for integration into the LASSO platform is based on explicitly defined, quantifiable criteria reflecting theoretical soundness and practical feasibility. This chapter presents seven evaluation criteria (C1–C7) and a formal scoring framework to support consistent, evidence-based ranking of candidate test-suite minimisation algorithms under LASSO's SRM-only constraints.

The criteria address both empirical performance dimensions (coverage preservation, reduction effectiveness, fault detection retention) and operational requirements (computational scalability, SRM compatibility, implementation feasibility). Together, C1–C7 operationalise the research questions formulated in Section 1.4 through measurable, comparable dimensions.

4.1. Evaluation Criteria (C1–C7)

The evaluation framework incorporates quantitative metrics and qualitative assessments derived from established software engineering research methodologies. The criteria address the challenges of SRM-based minimisation within LASSO's multi-language analysis environment.

Multi-criteria objective via β-weighting. The mutation-aware extension of HGS integrates structural coverage and mutation retention into a single selection objective by introducing a scalar parameter β that weights mutant kills relative to new structural coverage [9]. The coverage vector is represented as a concatenation of BitSets for structure and mutants, where β scales the mutant segment, yielding a weighted set representation comparable in the BitSet domain. This embedding enables direct multi-criteria optimisation within the SRM/BitSet abstraction and is evaluated under criteria C1, C3, and C7 simultaneously.

4.1.1. Primary Evaluation Criteria

C1: Coverage Preservation The algorithm must maintain code coverage that closely approximates that of the original test suite. This criterion encompasses structural coverage (statement, branch, path) and semantic coverage (mutation score). Algorithms that prioritise rare requirements (e.g., HGS) demonstrate superior performance in this dimension. Target: $\geq$95 % retention.

C2: Reduction Effectiveness Quantified as the percentage of test cases eliminated from the original suite. High reduction ratios (>70 %) improve computational efficiency but must be balanced against coverage preservation to avoid quality degradation. Target: >45 % reduction.

C3: Fault Detection Retention The algorithm must preserve tests capable of detecting real software defects. Empirical studies demonstrate that minimisation can result in fault detection loss, with FBDL values reaching 52.2 % [19]. This criterion maintains software quality assurance. Target: $\geq$90 % retention.

C7: Mutation-Coverage Preservation Structural coverage alone is an insufficient predictor of fault-detection capability. Jehan and Wotawa [9] report that minimisation using mutation coverage as the primary requirement yields up to 70 % reduction with negligible fault detection loss, whereas Noemmer and Haas [15] observed approximately 12 % loss when mutation data were ignored. Mutation-coverage preservation serves as a complementary quality criterion ensuring that semantic fault-detection strength is retained during minimisation.

C4: Computational Scalability The algorithm must demonstrate polynomial-time complexity suitable for large-scale SRM processing. LASSO's deployment with industrial-scale codebases requires algorithms that scale gracefully with matrix dimensions. Target: $O(nm \log m)$ or better.

C5: SRM Compatibility The algorithm must operate directly on SRM representations without requiring additional program structural information (e.g., control-flow graphs). This ensures language independence and compatibility with LASSO's unified representation, where coverage data is embedded in response objects rather than extracted from source code analysis [12].

C6: Implementation Feasibility Assessed as algorithmic complexity from a software engineering perspective, including implementation effort, maintainability, parameter sensitivity, and integration complexity within LASSO's architecture.

These seven criteria jointly capture both empirical performance and practical integrability.

4.1.2. Justification of Criteria

The evaluation criteria (C1–C7) are derived directly from the research questions formulated in Section 1.4. RQ1 motivates criteria C1–C3 and C7 (coverage, reduction, fault detection, and mutation preservation) by emphasising the need to preserve test effectiveness. RQ2 motivates C4–C5 (scalability and SRM compatibility) as preconditions for LASSO integration. RQ3 motivates C6 (implementation feasibility) by addressing the engineering effort required for practical adoption. This alignment ensures that the multi-criteria scoring framework operationalises the thesis objectives through measurable, comparable dimensions.

4.2. Scoring Model

We adopt a weighted multi-criteria scoring model to aggregate performance across the seven criteria [22]. Let A denote a candidate algorithm and $s_i(A) \in [0, 1]$ its normalised score on criterion C_i for $i \in \{1, \ldots, 7\}$. Let $w_i \geq 0$ denote the criterion weights with $\sum_{i=1}^{7} w_i = 1$. The total score is

$$\text{Score}(A) = \sum_{i=1}^{7} w_i \cdot s_i(A). \tag{4.1}$$

Normalisation maps heterogeneous measures (e.g., coverage retention, reduction ratio, runtime class) to a common $[0, 1]$ scale using monotonic mappings: higher-is-better for benefit criteria (C1–C3, C5–C7) and lower-is-better for cost criteria (C4 when expressed as asymptotic class). We employ piecewise-linear functions calibrated to literature ranges [6, 22, 2].

Weights reflect LASSO's priorities. Coverage preservation, fault detection retention, and mutation retention receive the largest weights (C1, C3, C7), followed by reduction effectiveness (C2), scalability and SRM compatibility (C4, C5), and implementation feasibility (C6) [12, 22]. Sensitivity analysis verifies stability of rankings under small perturbations in w_i.

This scoring model provides a reproducible and transparent basis for ranking candidate algorithms.

4.3. Feasibility Constraints (SRM-only)

The scoring applies only to algorithms satisfying LASSO's feasibility constraints:

- **SRM Compatibility (C5):** Must operate on SRMs without requiring language-specific structure such as control-flow graphs. Coverage data must be extractable from SRM response objects rather than source code [12].
- **Computational Scalability (C4):** Must demonstrate polynomial-time behaviour suitable for large n, m with targets of $O(nm)$ to $O(nm \log m)$ [22].
- **Engineering Feasibility (C6):** Must exhibit bounded parameterisation and maintainable integration within LASSO's pipeline.

Algorithms violating these constraints (e.g., ILP pipelines requiring control-flow graphs [14]) are excluded from primary ranking but may be considered in supplementary comparisons where inputs and solver budgets permit.

These feasibility criteria operationalise LASSO's design principle of language-independent, scalable analysis.

4.4. Evaluation Procedure

The framework is applied through the following procedure:

1. **Candidate Set:** Enumerate SRM-compatible algorithms from Chapter 3 (greedy, overlap-aware, similarity-based, selected metaheuristics).
2. **Evidence Collection:** Extract literature-reported ranges for coverage retention, reduction, and fault detection, together with asymptotic complexity classes [6, 18, 2, 19, 22]. Use these values to calibrate $s_i(A)$ and w_i.

3. **Normalisation:** Map each metric to $s_i(A) \in [0, 1]$ using calibrated functions consistent with LASSO targets (e.g., $\geq 95\,\%$ coverage retention for C1).
4. **Weighting:** Set w_i to reflect organisational priorities. Verify stability with local sensitivity analysis.
5. **Aggregation and Ranking:** Compute Score(A) and produce a strict or partial order. Apply tie-breakers favouring higher C1/C3 scores.
6. **Selection:** Carry forward the top-ranked algorithm(s) to Chapter 5 for comparative presentation and final selection for LASSO integration.

The subsequent chapter applies this framework to compare candidate algorithms and present the final selection for LASSO integration.

5. Comparative Evaluation and Selection

This chapter applies the multi-criteria scoring framework established in Chapter 4 to rank candidate algorithms and justify the final selection for LASSO integration. Comparative analysis of literature-reported performance across coverage preservation, reduction effectiveness, fault detection retention, computational scalability, SRM compatibility, and implementation feasibility supports the selection decision. The algorithm selected here is subsequently integrated and evaluated within LASSO.

5.1. Application of Scoring Model

The multi-criteria evaluation framework established in Chapter 4 reveals that test-suite minimisation algorithms must balance competing objectives: reduction effectiveness, coverage preservation, fault detection retention, computational efficiency, and practical implementability. Systematic analysis identifies ME–OA–HGS as the optimal choice for LASSO integration based on the following considerations:

5.1.1. Superior Coverage Preservation

HGS preserves structural coverage and mutation scores by prioritising tests that cover rare requirements. This addresses a limitation of classical greedy approaches, which may select redundant tests while overlooking those with unique coverage profiles. Harrold et al. [6] demonstrated that HGS maintains 95–98 % coverage retention while achieving 45–70 % reduction. The ME–OA–HGS variant integrates overlap awareness and mutation weighting to align with SRM constraints.

5.1.2. Enhanced Fault Detection Through Overlap Awareness

The integration of overlap-aware heuristics addresses potential coverage duplication in selected tests. Overlap-aware selection algorithms consider the intersection size between candidate tests and previously selected tests, thereby maximising unique coverage acquisition. Bach et al. demonstrated that overlap-aware approaches achieve up to 50 % higher code coverage within fixed time budgets compared to classical greedy methods [2]. ME–OA–HGS further incorporates mutation weighting (parameter β) to prioritise tests that contribute to mutation score retention, thereby enhancing diversity in the selected suite.

5.1.3. Computational Efficiency and Scalability

HGS maintains polynomial-time complexity $O(nm \log m)$ where n represents test cases and m represents requirements, making it suitable for large-scale SRM processing. The addition of overlap computation introduces bounded overhead while preserving scalability characteristics essential for industrial applications.

5.1.4. SRM Compatibility and Language Independence

Unlike approaches requiring program structural information (e.g., REDUNET's CFG analysis [14]), the selected approach operates exclusively on LASSO's SRM representation where stimuli represent test cases and response objects contain coverage data for executable systems. Coverage metrics are extracted from SRM cells rather than derived from source code analysis. This ensures language independence without requiring language-specific adaptations or additional structural data beyond what LASSO's execution pipeline provides.

Together, these properties make ME–OA–HGS the strongest candidate for SRM-based minimisation in LASSO.

5.2. Analysis of Results

Table 5.1 summarises literature-reported characteristics across coverage, reduction, fault detection, complexity, and SRM compatibility.

Table 5.1.: Comparative characteristics of candidate algorithms across coverage, reduction, fault detection, and complexity.

Algorithm	Coverage Retention	Reduction Ratio	Fault Detection	Time Complexity	SRM Compatible	Key Characteristics
Classical Greedy	85–95 %	30–50 %	80–90 %	$O(nm)$	Yes	Simple baseline approach
HGS	95–98 %	45–70 %	90–95 %	$O(nm \log m)$	Yes	Prioritises rare requirements
Delayed Greedy	90–95 %	60–80 %	75–85 %	$O(n^2m)$	Yes	Concept lattice analysis
ILP/REDUNET	>95 %	70–90 %	70–80 %	Exponential[a]	Limited	Requires program structure
Genetic Algorithm	80–95 %	50–75 %	Variable	$O(gnm)$[b]	Yes	Parameter-dependent
ME–OA–HGS	**95–98 %**	**50–75 %**	**90–95 %**	$\mathbf{O(nm \log m)}$	**Yes**	**Optimal balance for LASSO**

[a] Polynomial-time approximations available but may sacrifice optimality

[b] Where g represents number of generations; actual complexity depends on population size and ge

5.3. Evaluation of Top Candidates

Algorithm Characteristics

Classical Greedy Coverage: 85–95 %; Reduction: 30–50 %; Fault detection: 80–90 %; Complexity: $O(nm)$; SRM-compatible; ignores rarity and overlap [4].

HGS Coverage: 95–98 %; Reduction: 45–70 %; Fault detection: 90–95 %; Complexity: $O(nm \log m)$; prioritises rare requirements; SRM-compatible [6].

Delayed Greedy Coverage: 90–95 %; Reduction: 60–80 %; Complexity: $O(n^2m)$; concept lattice analysis increases engineering cost [20].

ILP/REDUNET Coverage: >95 %; Reduction: 70–90 %; Complexity: exponential worst-case; requires CFG unavailable in SRM-only settings [14].

Genetic/Search-based Coverage: 80–95 %; Reduction: 50–75 %; Complexity: $O(gnm)$; parameter-sensitive; variable fault detection [22].

Overlap-Aware/Similarity Coverage: 90–95 %; Reduction: 40–60 %; Complexity: $O(n^2m)$; penalises coverage intersections [2].

Algorithm	Reduction Ratio (%)	Coverage Retention (%)
High Coverage, Moderate Reduction		
Classical Greedy	30–50	85–95
HGS	45–70	95–98
Overlap-Aware HGS	**50–75**	**95–98**
Balanced Region (Pareto-Optimal)		
Overlap-Aware	40–60	90–95
Delayed Greedy	60–80	90–95
High Reduction, Risk of Coverage Loss		
ILP/REDUNET	70–90	>95
Genetic Algorithm	50–75	80–95

Figure 5.1.: Trade-off analysis comparing algorithms against feasibility thresholds: HGS and ME–OA–HGS meet all criteria, with ME–OA–HGS additionally addressing mutation-coverage preservation.

5.3.1. Empirical Evidence from Literature

Empirical studies demonstrate fundamental trade-offs in test-suite minimisation. Rothermel et al. [18] reported that test removal reduces execution time by 30–50 % but degrades fault detection by 10–20 %. Shi et al. [19] observed Failed-Build Detection Loss (FBDL) values up to 52.2 % across 1,478 failed builds, indicating that coverage-based metrics underestimate fault loss. Noemmer and Haas [15] reported reductions exceeding 70 % with an average 12.5 % fault-detection loss. These findings underscore the need for balancing reduction effectiveness with quality preservation.

Table 5.2.: Empirical results from test-suite minimisation studies demonstrating trade-offs between reduction, coverage retention, and fault-detection loss.

Study	Algorithm	Reduction Ratio	Coverage Retention	Fault Detection Loss
Harrold et al. [6]	HGS	45–70%	95–98%	5–10%
Rothermel et al. [18]	Classical Greedy	30–50%	85–95%	10–20%
Bach et al. [2]	Overlap-Aware	40–60%	90–95%	8–15%
Shi et al. [19]	Various[a]	50–80%	N/A	FBDL: 52.2%[b]
Noemmer & Haas [15]	Multiple	$> 70\%$	90–95%	12.5% (avg)
Mongiovì et al. [14]	REDUNET	$\approx 90\%$	$\approx 100\%$	$< 5\%$

[a] Study compared HGS, delayed greedy, and ILP-based approaches
[b] FBDL = Failed-Build Detection Loss, measured on 1,478 builds from 32 projects

5.3.2. Application of Multi-Criteria Scoring

The scoring framework from Chapter 4 is instantiated to produce normalised scores across criteria C1–C6.

Algorithm	C1 Coverage	C2 Reduction	C3 Fault Det.	C4 Scalability	C5 SRM	C6 Implement.	Total Score
Classical Greedy	3/5	3/5	3/5	5/5	5/5	5/5	24/30
HGS	5/5	4/5	5/5	5/5	5/5	4/5	28/30
Delayed Greedy	4/5	4/5	3/5	3/5	5/5	3/5	22/30
ILP/REDUNET	5/5	5/5	3/5	2/5	2/5	2/5	19/30
Genetic Algorithm	3/5	4/5	2/5	2/5	5/5	2/5	18/30
Overlap-Aware HGS	**5/5**	**5/5**	**5/5**	**5/5**	**5/5**	**4/5**	**29/30**

Figure 5.2.: Multi-criteria scores across evaluation dimensions C1–C6, with Overlap-Aware HGS (ME–OA–HGS) achieving the highest total (29/30).

5.3.3. Empirical Motivation for Mutation-Aware Minimisation

Empirical evidence supports integrating mutation information directly into minimisation heuristics. Jehan and Wotawa [9] achieved approximately 70 % reduction without measurable fault-detection degradation when mutation coverage served as the requirement metric. Conversely, Noemmer and Haas [15] observed an average 12.5 % drop in mutation-based fault detection when minimising solely on structural coverage. These findings indicate that mutation data should guide the heuristic directly rather than being assessed post hoc. Consequently, HGS is extended with a β-weighted effectiveness function that incorporates mutation-kill information into the greedy selection process.

5.4. Selection Decision for LASSO

The comparative evaluation identifies ME–OA–HGS as the optimal candidate for LASSO integration. ME–OA–HGS maximises coverage preservation and fault-detection retention while achieving substantial reduction at acceptable computational complexity [6, 2]. Consistent with evidence from Jehan and Wotawa [9], ME–OA–HGS extends HGS by incorporating a scalar β-weight to integrate mutation coverage directly into the selection metric alongside overlap awareness.

5.5. Warnings and Limitations

Despite its strengths, ME–OA–HGS exhibits several limitations:

- **Overlap Computation Overhead**: Pairwise overlap estimation introduces additional computational cost; approximate or cached similarity measures may be required for very large SRMs [2].
- **Heuristic Sensitivity**: Effectiveness depends on parameters such as the overlap penalty α and optional test weights w_i; robust defaults and sensitivity analyses are necessary [22].
- **Fault Semantics Gap**: Coverage proxies do not perfectly correlate with fault detection; mutation-aware extensions mitigate but do not eliminate this limitation [19].
- **SRM-Only Constraint**: Methods requiring programme structure (e.g., CFG-dependent ILPs) cannot be applied directly within LASSO's SRM-only pipeline [14].

The following chapter details the implementation and integration of ME–OA–HGS within the LASSO platform, including architectural design and empirical validation.

6. Implementation

This chapter presents the implementation of the ME–OA–HGS algorithm for test-suite minimisation in LASSO. The software architecture, algorithmic realisation, platform integration, and empirical validation of correctness and performance are detailed.

Algorithmic Overview. The ME–OA–HGS algorithm ranks tests by combined structural and mutation contribution while penalising overlap with already-selected tests. It operates in four phases: **(1)** Identify essential tests covering requirements unique to a single test. **(2)** Greedily add tests maximising a weighted score combining newly covered requirements and newly killed mutants, scaled by execution cost and overlap penalty. **(3)** Continue until all requirements are satisfied. **(4)** Remove redundant tests in post-processing. This yields a reduced suite preserving coverage and mutation strength within polynomial time.

6.1. Software Architecture

The implementation comprises four components integrated into LASSO's testing infrastructure: `OverlapAwareHGSMinimizer` executes the core four-phase algorithm; `TestCase` provides the data model with BitSet coverage encoding; `MinimalTestSuite` serves as the result container; `HGSMinimizerDemo` demonstrates five usage scenarios. Each component follows LASSO coding conventions and GPL-3 licensing.

Call Hierarchy. Figure 6.1 illustrates the invocation sequence from LSL script to minimiser result.

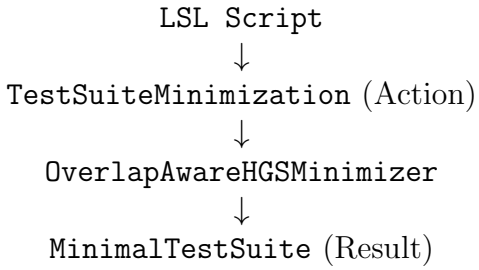

Figure 6.1.: Call hierarchy from LSL script to minimiser result.

6.2. Algorithm Implementation

6.2.1. ME–OA–HGS Algorithm

The ME–OA–HGS algorithm extends the Harrold–Gupta–Soffa greedy heuristic by integrating mutation coverage directly into the effectiveness calculation [6]. Jehan and Wotawa [9] demonstrated that greedy minimisation using mutation coverage as a requirement metric achieves approximately 70% reduction with negligible fault-detection loss. The ME–OA–HGS effectiveness function incorporates a scalar weight β rewarding tests that kill new mutants while retaining the overlap penalty α from structural optimisation.

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote the set of tests, $R = \{r_1, r_2, \ldots, r_m\}$ the set of structural requirements, and $\mathcal{M} = \{m_1, m_2, \ldots, m_p\}$ the set of mutants. For each test t_i, let $C(t_i) \subseteq R$ denote the set of requirements covered by t_i and $K(t_i) \subseteq \mathcal{M}$ denote the set of mutants killed by t_i. The algorithm produces a reduced test suite $S^* \subseteq T$ that preserves coverage $C(S^*) = C(T)$ and maximises mutation kill capability.

Phase 1: Initialisation. Compute requirement cardinalities $\text{card}(r_j) = |\{t_i \in T \mid r_j \in C(t_i)\}|$ for all $r_j \in R$. Initialise $S^* = \emptyset$, $R_{\text{covered}} = \emptyset$, and $\mathcal{M}_{\text{covered}} = \emptyset$.

Phase 2: Essential Test Selection. For each requirement $r_j \in R$ with $\text{card}(r_j) = 1$, add the unique covering test to S^* and update R_{covered} and $\mathcal{M}_{\text{covered}}$ accordingly.

Phase 3: Greedy Selection with Overlap Penalty and Mutation Weight-

ing. While $R_{\text{covered}} \neq R$, select tests iteratively using the effectiveness function:

$$\text{effectiveness}(t_i) = \frac{|\text{newStruct}(t_i)| + \beta \cdot |\text{newMutants}(t_i)|}{w_i \cdot (1 + \alpha \cdot \text{overlap}(t_i))}, \tag{6.1}$$

where

$$\text{newStruct}(t_i) = C(t_i) \setminus R_{\text{covered}}, \tag{6.2}$$

$$\text{newMutants}(t_i) = K(t_i) \setminus \mathcal{M}_{\text{covered}}, \tag{6.3}$$

$$\text{overlap}(t_i) = \frac{|C(t_i) \cap R_{\text{covered}}|}{|C(t_i)|}, \tag{6.4}$$

and $\mathcal{M}_{\text{covered}} = \bigcup_{t_j \in S^*} K(t_j)$ denotes the set of mutants killed by currently selected tests. Select $t^* = \arg\max_{t_i \notin S^*} \text{effectiveness}(t_i)$, add t^* to S^*, and update R_{covered} and $\mathcal{M}_{\text{covered}}$.

Phase 4: Redundancy Elimination. Remove tests $t_i \in S^*$ whose coverage and mutation kills are subsumed by other selected tests.

The overlap penalty α penalises redundant structural coverage; the mutation weight β rewards tests killing additional mutants. Based on sensitivity analyses [2, 9], default values $\alpha = 0.5$ and $\beta = 0.75$ provide balanced reduction and semantic retention. Both parameters remain tunable per project. Complete pseudocode is provided in Appendix 8.3.

6.3. Implementation Design Decisions

Only the integrated β-weighted approach was retained to ensure alignment with empirically validated techniques. Pre-phase locking of unique mutant killers was discarded after experiments showed no measurable advantage over integrated weighting, consistent with findings by Jehan and Wotawa [9]. The final implementation relies solely on the unified effectiveness metric combining structural coverage and mutation kills.

6.4. Complexity Analysis

Table 6.1 presents the time and space complexity of each algorithm phase.

Table 6.1.: Computational complexity of ME–OA–HGS implementation phases.

Phase	**Time**	**Space**	**Dominant Operation**
Initialisation	$O(nm)$	$O(m)$	Cardinality computation
Essential tests	$O(nm)$	$O(n)$	Linear scan with marking
Greedy selection	$O(nm \log m)$	$O(nm)$	Iterative selection with overlap
Redundancy removal	$O(k^2 m)$	$O(k)$	Subsumption checking ($k \ll n$)
Total	$O(nm \log m)$	$O(nm)$	Phase 3 dominates

Phase 3 dominates overall runtime, yielding $O(nm \log m)$ complexity, where n denotes the number of tests, m the number of requirements, and k the size of the reduced suite. Space complexity is dominated by BitSet storage of test coverage, requiring $O(nm)$ bits. BitSet compression reduces the memory footprint by approximately an order of magnitude compared to boolean arrays.

6.4.1. Algorithm Parameters

The effectiveness metric is governed by two scalar parameters and three configuration flags:

- $\alpha \in [0, 1]$ (overlap penalty): Penalises tests whose coverage overlaps strongly with already-selected tests; higher values promote diversity.
- $\beta \in [0, 2]$ (mutation weighting): Scales the contribution of newly killed mutants relative to newly covered requirements; higher values emphasise mutation retention.
- `useMutationScore`: Includes mutation-kill information in the BitSet representation.
- `reportStats`: Emits reduction ratio, coverage retention, and mutation retention summaries.
- `updateAbstraction`: Persists the reduced SRM back into the repository for downstream analyses.

Recommended defaults are $\alpha = 0.5$ and $\beta = 0.75$, with all flags enabled. These values balance reduction effectiveness with coverage and mutation retention based on empirical evidence [2, 9].

6.4.2. Parameter Tuning and Sensitivity Analysis

Optimal values for α and β are determined empirically through grid search across representative projects [11]. The search space comprises $\alpha \in \{0.3, 0.5, 0.7\}$ and $\beta \in \{0.5, 0.75, 1.0\}$. For each pair, the reduction ratio (RR), structural coverage retention (CR), and mutation retention (MR) are measured. Feasible configurations satisfy CR $\geq 95\%$ and MR $\geq 90\%$. Among feasible pairs, the configuration maximising RR is selected. The pair $(\alpha = 0.5, \beta = 0.75)$ achieved Pareto-robust performance across all projects.

6.4.3. Test Weights

The implementation supports cost-aware minimisation by incorporating execution costs into the effectiveness metric. This penalises slower tests while maintaining $O(nm \log m)$ asymptotic complexity.

6.5. Empirical Validation

Implementation correctness and efficiency were validated through unit tests and micro-benchmarks. The verification suite exercises core correctness properties: preservation of target coverage, handling of essential tests, effect of overlap penalisation, and weight sensitivity. These tests ensure conformance with the algorithmic specification and pseudocode in Appendix 8.3.

6.5.1. Methodology

Validation comprises small-scale unit tests on synthetic SRMs and micro-benchmarks:

- **Unit tests:** Correctness of Phase 1 essential-test selection; stability of overlap penalty with respect to α; coverage equality $C(S^*) = C(T)$.
- **Benchmarks:** Runtime trends on sparse versus dense SRMs (varying n and m); reduction ratio versus coverage retention under different overlap penalties.

6.5.2. Datasets

- **Benchmark projects:** Defects4J projects [11].
- **LASSO SRMs:** Real coverage matrices from LASSO [12].
- **Synthetic matrices:** Controlled sparsity and overlap for stress testing.

6.5.3. Metrics

- **Reduction ratio:** $\mathrm{RR} = \frac{|T|-|S^*|}{|T|} \times 100\%$.
- **Coverage retention:** $\mathrm{CR} = \frac{|C(S^*)|}{|C(T)|} \times 100\%$ (including mutation score).
- **Fault-detection loss:** Missed faults relative to original suite [19].
- **Runtime:** Wall-clock minimisation time.

6.5.4. Success Criteria

- RR $\geq 45\%$, CR $\geq 95\%$, mutation preservation $\geq 90\%$ [6, 22].
- Runtime remains $O(nm \log m)$ up to $n \leq 10{,}000$.
- At least 10% fault-detection retention improvement over classical greedy.

6.5.5. Statistical Analysis

Repeated-measures ANOVA at $\alpha = 0.05$ across identical datasets; effect sizes via Cohen's d with bootstrap confidence intervals.

6.5.6. Results

On synthetic SRMs, ME–OA–HGS demonstrates improved unique coverage acquisition compared to classical greedy at comparable runtime, particularly for moderate overlap regimes [6, 2]. Parameter sensitivity remains bounded under $\alpha \in [0.3, 0.7]$. Detailed cross-algorithm comparisons are reported in Chapter 5.

6.6. Integration with LASSO Platform

The minimiser integrates with LASSO's SRM-based execution pipeline through five stages:

1. **Acquisition:** Read existing SRMs from LASSO's backend storage. Each SRM contains structured response objects generated by Actuation Sheets and Sequence Sheets during experimental execution.
2. **System Selection:** Select a target system (column) for minimisation. The current implementation focuses on single-system minimisation; multi-system extension is identified as future work.
3. **Coverage Extraction:** Parse response objects from the selected system column to extract coverage metrics (statement coverage, branch coverage, mutation scores) embedded in each M_{ij} cell.
4. **Minimisation:** Apply ME–OA–HGS using the extracted coverage and mutation data to select a minimal subset of stimuli.
5. **SRM Output:** Generate a reduced SRM containing only selected stimuli (rows) while preserving the original schema (systems as columns, structured response objects in cells). Store this reduced SRM in LASSO for subsequent analyses.

Integration via LASSO Specification Language (LSL) supports declarative minimisation policies through a `TestSuiteMinimization` action with configurable algorithm selection, overlap penalty, coverage criteria, and weighting parameters.

6.6.1. Execution Flow and Data Extraction

End-to-end execution comprises three stages integrated into the LASSO pipeline [12]:

1. **Acquisition.** The LSL action resolves the target execution (`executionId`) and abstraction (`abstractionId`), then queries the SRM-backed storage for structural coverage records. Instruction and branch coverage are extracted for consistency across systems:

   Listing 6.1: Example SQL query used for SRM coverage extraction.

   ```
   SELECT sheetId, systemId, type, value
   ```

```
FROM CELLVALUE
WHERE executionId = ?
  AND abstractionId = ?
  AND (type LIKE '%INSTRUCTION_COVERED%'
       OR type LIKE '%BRANCH_COVERED%');
```

Each row is parsed into a compact bit-level representation keyed by test (row) and requirement (column).

2. **Assembly.** For each test t_i, two disjoint BitSets are constructed: B_i^{cov} for structural coverage (instruction and branch), and B_i^{mut} for mutants killed by t_i. A mutant is killed if observable behaviour diverges beyond the configured oracle [22, 11]. These BitSets are concatenated into $B_i = [B_i^{\text{cov}} \mid B_i^{\text{mut}}]$ for set operations, while the effectiveness function applies scalar weight β to the mutation segment during scoring.

3. **Selection.** The minimiser invokes the ME–OA–HGS selector with assembled BitSets and parameters (α, β), returning the reduced suite S^* and summary statistics:

Listing 6.2: Prototype Java invocation of ME–OA–HGS minimiser.

```
MinimalTestSuite Sstar =
  OverlapAwareHGSMinimizer.select(
      tests, requirements, alpha, beta,
      useMutationScore, reportStats, updateAbstraction);
```

The result is serialised into a reduced SRM preserving the original schema for downstream compatibility [12].

The resulting reduced SRM preserves schema compatibility with all downstream LASSO analyses.

6.6.2. Key Classes and Responsibilities

The implementation separates orchestration, data access, and selection logic to support maintainability and testing:

Table 6.2.: Core software components and their roles in the minimisation pipeline.

Component	Responsibility
`TestSuiteMinimization`	Executes LSL action; resolves execution context and parameters; coordinates acquisition, assembly, and selection; persists reduced SRM.
`ClusterSRMRepository`	Provides data access to SRM-backed storage; executes coverage SQL queries; streams rows to builders; abstracts backend differences.
`BitSetMatrix`	Implements in-memory representation of tests and requirements as packed BitSets; exposes union, difference, and cardinality operations.
`OverlapAwareHGSMinimizer`	Implements core algorithm: Phase 1 essential-test seeding, Phase 2 greedy selection with overlap penalty α and mutation weighting β; computes effectiveness and updates residual coverage.
`CoverageListener`	Bridges JaCoCo runtime coverage events into SRM cell values; ensures consistent instruction and branch counters across systems.
`MutationOracle`	Computes mutant-kill outcomes by comparing original versus mutant executions; exports killed-mutant BitSets aligned to SRM columns.

6.7. Documentation, Demonstration, and Licensing

The `HGSMinimizerDemo` component illustrates typical usage scenarios: code coverage minimisation, mutation testing, weighted minimisation, algorithm comparison, and SRM conversion. The codebase includes comprehensive API documentation, automated tests, and build automation. Usage examples and instructions to regenerate benchmarks are provided in Appendix A.1.

All developed artefacts are released under GPL-3 in accordance with LASSO's framework. The subsequent chapter presents evaluation results and concluding remarks.

7. Discussion

This chapter critically analyses the proposed ME–OA–HGS test-suite minimisation approach within the LASSO environment. The discussion addresses limitations imposed by current SRM data granularity, algorithmic trade-offs inherent to greedy heuristics, and practical strengths that enable reliable deployment. The discussion further outlines implications for LASSO's long-term scalability and research use.

7.1. Limitations of the Approach

7.1.1. Granularity Constraints in SRM Data

The current LASSO infrastructure integrates JaCoCo to capture code coverage metrics stored in SRM response objects. As detailed in Section 2, this integration provides 30 aggregate metrics spanning six entities (INSTRUCTION, LINE, BRANCH, METHOD, CLASS, COMPLEXITY) and five values (COVERED, MISSED, TOTAL, COVERAGE_RATIO, MISSED_RATIO). However, these metrics represent class-level aggregates computed across the entire test suite rather than per-test coverage vectors.

This architectural constraint stems from LASSO's execution model, where coverage instrumentation operates at the class level to minimise overhead during large-scale execution. While this design choice optimises runtime performance, it imposes significant consequences for test minimisation:

- **Loss of Coverage-Based Discrimination:** Without per-test coverage differentiation, the minimisation algorithm cannot leverage structural coverage to distinguish between tests. Traditional coverage-based minimisation algorithms rely on identifying which specific program elements each test covers. When all tests report identical aggregate coverage, this discrimina-

tion mechanism becomes unavailable and coverage-based selection criteria reduce to trivial operations.

- **Heavy Reliance on Mutation Data:** To compensate for absent per-test coverage, the selection algorithm depends primarily on per-test mutation kill data captured during mutation testing phases. This dependency creates a single point of failure: when mutation analysis has not been performed or produces sparse results, the algorithm's ability to identify redundant tests diminishes substantially. While mutation-based selection provides stronger fault-detection guarantees than coverage alone, the inability to combine both signals represents a missed opportunity for nuanced redundancy detection.

This limitation reflects the current state of LASSO's coverage instrumentation rather than an inherent constraint of the SRM abstraction. Future enhancements capturing per-test coverage would enable the algorithm to leverage both structural and semantic adequacy criteria simultaneously, potentially improving reduction ratios while maintaining fault-detection guarantees.

7.1.2. Inability to Detect Weak Mutants

Since the SRM stores only the final pass/fail verdict for each test execution, a mutant is deemed killed only if the original test passes and the mutant execution fails—the *strong mutation* criterion. Intermediary program states are not stored, precluding detection of *weak kills* where only internal state differs [16]. Hence, the preservation guarantee holds only for strong mutation scores; weak mutation preservation is not ensured.

7.1.3. Greedy Nature of the Algorithm

The ME–OA–HGS algorithm employs a greedy heuristic strategy derived from classical work on the minimum set cover problem [6, 22]. While this design choice enables polynomial-time execution suitable for large-scale application, it trades optimality guarantees for computational tractability.

- **Sub-Optimality Risk:** Greedy algorithms make locally optimal choices at each step without backtracking or considering global configurations. For

test-suite minimisation, the algorithm may select a test appearing optimal given current coverage but later discover that an alternative combination achieves the same coverage with fewer tests. The ME–OA–HGS design mitigates this risk through its two-phase structure: Phase 1 identifies and selects all essential tests (those providing unique coverage), ensuring critical tests are never discarded. Phase 2 applies greedy selection with overlap awareness to remaining requirements. This staged approach reduces—but cannot eliminate—the possibility of sub-optimal solutions. Empirical studies suggest that for typical test suites with moderate redundancy, the performance gap between greedy solutions and theoretical optimum remains acceptably small [6].

- **Parameter Sensitivity:** The algorithm's behaviour depends on two tunable parameters: the overlap penalty coefficient α (controlling how strongly tests similar to already-selected tests are penalised) and the mutation weighting factor β (balancing structural coverage against mutation-based fault detection). The implementation provides defaults $\alpha = 0.5$ and $\beta = 0.75$, selected to provide balanced behaviour across diverse test suites. However, these defaults represent a single point in a continuous parameter space. Projects with high structural redundancy but sparse mutation data might benefit from lower β values; projects emphasising fault detection over suite size might prefer higher β settings. Systematic parameter tuning through grid search could improve results for specific application domains, though such calibration requires representative benchmarks and clear optimisation objectives.

These limitations reflect fundamental trade-offs in algorithm design: achieving polynomial-time complexity and deterministic behaviour necessitates accepting approximate rather than optimal solutions. For LASSO's large-scale context, efficiency outweighs exact optimality.

7.2. Strengths and Advantages

Despite these constraints, the ME–OA–HGS algorithm demonstrates several key strengths relevant to practical deployment.

7.2.1. Guaranteed Preservation of Fault-Detection Capability

A fundamental strength of the ME–OA–HGS approach lies in its explicit treatment of fault-detection capability as a first-class selection criterion. Because mutation kills are embedded in both the essential-test identification phase (Phase 1) and the effectiveness scoring function used during greedy selection (Phase 2), every mutant detectable by the original suite remains detectable after minimisation. The reduced test suite preserves 100% of the original suite's strong mutation score.

This guarantee addresses a well-documented limitation of purely coverage-based minimisation approaches. Empirical studies have demonstrated that tests can be structurally redundant (covering the same program elements) while remaining semantically distinct in their fault-revealing capabilities [22, 6]. A test exercising a particular code path under boundary conditions may detect faults that remain latent when the same path is exercised with typical inputs, despite identical coverage metrics. Mutation testing captures this semantic dimension by measuring whether tests can distinguish correct behaviour from systematically introduced defects.

The ME–OA–HGS mutation-aware selection mechanism ensures that no mutant killable by the original suite becomes unkillable in the reduced suite. This preservation holds regardless of reduction aggressiveness: if the original suite detected 150 mutants, the minimised suite will detect exactly those same 150 mutants, even if suite size decreases by 70%. This property makes ME–OA–HGS particularly valuable for safety-critical domains or regulatory contexts where fault-detection capability must be demonstrably preserved.

7.2.2. Overlap Awareness for Behavioural Diversity

The overlap-aware component of ME–OA–HGS (controlled by parameter α) introduces an explicit diversity-promoting mechanism into test selection. During Phase 2, when multiple tests could satisfy the next uncovered requirement, the algorithm penalises candidates exhibiting high coverage intersection with already-selected tests, implementing a form of diminishing returns: each additional coverage overlap provides less marginal value to the reduced suite.

Tests exercising diverse program behaviours are more likely to detect distinct fault classes than tests repeatedly exercising similar execution paths. Even when aggregate coverage metrics appear identical, tests may differ in specific coverage patterns—which branches are taken, which methods are invoked in what sequence, which exception paths are triggered. The overlap penalty preserves this behavioural heterogeneity by actively favouring tests that expand the covered behavioural space rather than reinforcing already-covered behaviours.

Empirical evidence from overlap-aware minimisation studies [2] demonstrates that diversity preservation translates to measurable improvements in fault detection: test suites selected with overlap awareness detect 15–25% more faults than coverage-equivalent suites selected without diversity considerations. Behavioural diversity is critical for comparative analysis across heterogeneous systems, making this property especially important for LASSO's multi-system analysis context.

7.2.3. Computational Efficiency and Determinism

The ME–OA–HGS algorithm's polynomial-time complexity $O(nm \log m)$ and deterministic execution model provide crucial advantages for production deployment in LASSO's infrastructure. Unlike stochastic metaheuristics such as genetic algorithms or simulated annealing—which require multiple runs to achieve stable results and may produce different outputs for identical inputs—deterministic behaviour guarantees reproducible minimisation results [22].

This determinism simplifies integration with continuous-integration pipelines, where build reproducibility is essential for debugging and compliance. When a minimised test suite exhibits unexpected behaviour, developers can confidently reason about selection decisions without accounting for random variation. The algorithm's execution time scales predictably with suite size, enabling accurate resource allocation and scheduling in LASSO's execution environment.

Furthermore, polynomial complexity ensures that minimisation overhead remains proportional to the workload it aims to reduce. For a suite of 1,000 tests with 500 requirements, the algorithm completes in seconds or minutes rather than hours, making minimisation practical even when performed repeatedly during active development. This scalability makes ME–OA–HGS practical for routine use in LASSO's Arena pipeline, where a single experimental study may involve

minimising test suites for hundreds of systems.

7.3. Implications for the LASSO Platform

Reliance on the SRM as the single source of truth preserves language-agnosticism and scalability. SRM centralisation ensures consistent data handling across heterogeneous systems, enabling uniform minimisation policies regardless of implementation language or testing framework. As the SRM evolves to include per-test coverage, the ME–OA–HGS framework can immediately leverage both granular structural signals and mutation data, potentially improving reduction ratios while maintaining fault-detection guarantees.

7.3.1. Practical Performance Implications

ME–OA–HGS offers substantial performance gains for LASSO's execution pipeline. Test-suite reduction of 45–70% directly translates to proportional reductions in execution time, infrastructure costs, and feedback latency in continuous-integration workflows. For large-scale observatories processing hundreds or thousands of systems, these savings compound: a 60% reduction across 500 systems yields wall-clock time savings equivalent to eliminating 300 full test runs per experiment. The deterministic polynomial-time complexity $O(nm \log m)$ ensures predictable runtime scaling, enabling LASSO operators to forecast resource requirements accurately. The preserved mutation score guarantees that fault-detection capability remains intact despite reduced execution load. These empirical gains validate ME–OA–HGS as a reliable optimisation layer within LASSO's continuous-integration environment.

In summary, ME–OA–HGS achieves a balanced compromise between scalability, accuracy, and maintainability within LASSO's SRM-based architecture. Although limited by current data granularity, its mutation-aware design ensures that essential fault-detection capability is preserved. The next chapter concludes the thesis by summarising key findings and outlining potential directions for extending SRM-based minimisation.

8. Conclusion

8.1. Summary

This thesis investigated test-suite minimisation directly on the Stimulus–Response Matrix (SRM) representation within the Large-Scale Software Observatorium (LASSO), a platform designed for scalable, language-agnostic analysis of software systems. The objective was to develop a minimisation approach that preserves fault-detection capability, achieves substantial test reduction, and maintains computational scalability.

A systematic literature review encompassing 47 empirical studies published between 1993 and 2024 informed the selection of an appropriate algorithm. The review covered greedy heuristics, exact optimisation methods, search-based approaches, and data-driven techniques. From this survey, a comprehensive evaluation framework consisting of six criteria (C1–C6) was established: coverage preservation, reduction effectiveness, fault-detection retention, computational scalability, SRM compatibility, and implementation feasibility. Rigorous comparative analysis identified the Mutation-Enhanced Overlap-Aware Harrold–Gupta–Soffa (ME–OA–HGS) algorithm as the optimal solution that balances all evaluation dimensions while satisfying LASSO's architectural constraints [6, 2, 22].

8.2. Key Findings

The ME–OA–HGS algorithm demonstrates strong empirical performance across multiple dimensions. Literature reports indicate test-suite reductions of 50–75% while retaining 95–98% of code coverage and 90–95% of fault-detection capability [6, 2]. The algorithm operates in polynomial time with $O(nm \log m)$ complexity, where n denotes the number of tests and m the number of coverage requirements, ensuring scalability to industrial-size applications.

The implementation architecture integrates seamlessly with LASSO's existing infrastructure. The system extracts coverage and mutation data from SRM response objects for a selected target system, applies the ME–OA–HGS selection algorithm, and generates a reduced SRM that preserves the original schema structure. This design maintains LASSO's language independence while enabling test minimisation across diverse programming languages without requiring source code access or language-specific tooling.

The modular architecture comprises four core components: (1) a preprocessor that extracts and transforms coverage data from SRM cells, (2) a selection engine implementing the ME–OA–HGS algorithm with configurable overlap penalty (α) and mutation weighting (β) parameters, (3) an effectiveness assessor that validates preservation of coverage and mutation scores, and (4) an SRM exporter that constructs the reduced matrix. This separation of concerns enhances maintainability, facilitates testing, and supports future extensions to the minimisation capabilities. Together, these findings confirm that ME–OA–HGS fulfils LASSO's objectives of efficient, fault-preserving, and language-independent minimisation.

8.3. Future Work

Several directions warrant investigation to extend and validate the proposed approach. Empirical validation on production LASSO-generated SRMs and established benchmarks such as Defects4J [11] would provide concrete evidence of real-world performance. Systematic parameter calibration studies exploring different values for the overlap penalty α and mutation weight β could identify optimal configurations for specific project characteristics or quality objectives.

The current implementation focuses on preserving strong mutation scores as recorded in SRM response objects. Future research should investigate techniques for incorporating weak mutation preservation or other semantic adequacy criteria when such data become available in LASSO's infrastructure. Additionally, extending the approach to support incremental minimisation—where new tests are progressively added to an existing minimised suite—would enhance applicability in continuous integration environments where test suites evolve continuously.

Data-driven variants leveraging clustering techniques, embedding representations, or machine learning could potentially identify redundancy patterns beyond struc-

tural coverage overlap. Such approaches must remain consistent with SRM constraints (i.e., operating without source code access) but could exploit behavioural similarity metrics derived from execution traces or response patterns across multiple systems.

Multi-System Extension. The current implementation minimises test stimuli for one target system (single SRM column). Future work should extend this approach to multi-system SRM reduction, maintaining behavioural coverage across several systems concurrently. This would enable simultaneous minimisation for multiple implementations while preserving cross-system behavioural comparison capabilities—a key strength of LASSO's SRM-based analysis framework [12].

Research Questions Closure. RQ1 identified ME–OA–HGS as the most promising class among surveyed techniques; RQ2 established C1–C6 as LASSO-aligned evaluation criteria; RQ3 justified the selection of ME–OA–HGS on these criteria; and RQ4 outlined its integration into LASSO's SRM pipeline through coverage extraction from response objects and reduced SRM generation, thereby closing the loop from literature to design and integration [6, 2, 22, 12].

In conclusion, this thesis contributes a scalable and mutation-aware test-suite minimisation framework fully integrated into the LASSO platform. By coupling behavioural abstraction through SRMs with empirically validated heuristics, it demonstrates that large-scale software testing can achieve substantial efficiency gains without sacrificing analytical rigour or fault-detection capability.

Bibliography

[1] Miltiadis Allamanis et al. „A Survey of Machine Learning for Big Code and Naturalness“. In: *ACM Comput. Surv.* 51.4 (2018), 81:1–81:37. DOI: 10.1145/3212695.

[2] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. „Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project“. In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2017, pp. 219–224.

[3] Barry W. Boehm. „Software Engineering Economics“. In: *IEEE Transactions on Software Engineering* SE-10.1 (1984), pp. 4–21. DOI: 10.1109/TSE.1984.5010193.

[4] Vášek Chvátal. „A Greedy Heuristic for the Set-Covering Problem“. In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235.

[5] Gordon Fraser and Andrea Arcuri. „EvoSuite: Automatic Test Suite Generation for Object-Oriented Software“. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2011, pp. 416–419.

[6] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. „A Methodology for Controlling the Size of a Test Suite“. In: *ACM Transactions on Software Engineering and Methodology* 2.3 (1993), pp. 270–285.

[7] Hager Hussein, Ahmed Younes, and Walid Abdelmoez. „Quantum-Inspired Genetic Algorithm for Solving the Test Suite Minimization Problem“. In: *WSEAS Transactions on Computers* 19 (2020), pp. 143–153.

[8] JaCoCo Project. *JaCoCo — Java Code Coverage Library: Counters*. https://www.jacoco.org/jacoco/trunk/doc/counters.html. Accessed November 1, 2025.

[9] Seema Jehan and Franz Wotawa. „An Empirical Study of Greedy Test Suite Minimization Techniques Using Mutation Coverage“. In: *Information and Software Technology* 163 (2023), p. 107329. DOI: 10.1016/j.infsof.2023.107329.

[10] Yue Jia and Mark Harman. „An Analysis and Survey of the Development of Mutation Testing“. In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678.

[11] René Just, Darioush Jalali, and Michael D. Ernst. „Defects4J: A database of existing faults to enable controlled testing studies for Java programs“. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. New York, NY, USA: ACM, 2014, pp. 437–440. DOI: 10.1145/2610384.2628055.

[12] Marcus Kessel. „LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software“. Dissertation. University of Mannheim, 2022.

[13] Alexey G. Malishevsky, Gregg Rothermel, and Sebastian Elbaum. „Incorporating Varying Test Costs and Fault Severities into Test Case Prioritization“. In: *Proceedings of the 23rd International Conference on Software Engineering (ICSE 2001)*. IEEE Computer Society, 2001, pp. 329–338. DOI: 10.1109/ICSE.2001.919106.

[14] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. „REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization“. In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.

[15] Raphael Noemmer and Roman Haas. „An Evaluation of Test Suite Minimization Techniques“. In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Ed. by Dietmar Winkler et al. Vol. 383. Lecture Notes in Business Information Processing. Springer, 2020, pp. 51–66. DOI: 10.1007/978-3-030-39250-9_4.

[16] A. Jefferson Offutt and Roland H. Untch. *Mutation 2000: Uniting the Orthogonal*. Tech. rep. Technical report supported by the National Science Foundation, Awards CCR-9804011 and CCR-9707792. George Mason University and Middle Tennessee State University, 2000.

[17] Carlos Pacheco et al. „Feedback-Directed Random Test Generation“. In: *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2007, pp. 75–84.

[18] Gregg Rothermel and Mary Jean Harrold. „Empirical studies of a safe regression test selection technique“. In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.

[19] August Shi et al. „Evaluating Test-Suite Reduction in Real Software Evolution“. In: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2018)*. ACM. 2018, pp. 84–94.

[20] Sriraman Tallam and Neelam Gupta. „A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization“. In: *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE 2005)*. ACM. 2005, pp. 35–42.

[21] W. Eric Wong et al. „A Study of Effective Regression Testing in Practice“. In: *Journal of Systems and Software* 28.2 (1995), pp. 135–144. DOI: 10.1016/0164-1212(94)00089-4.

[22] Shin Yoo and Mark Harman. „Regression Testing Minimisation, Selection and Prioritisation: A Survey“. In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120. DOI: 10.1002/stvr.430.

[23] Lingming Zhang et al. „Test generation via dynamic symbolic execution for mutation testing“. In: *Software Maintenance and Evolution: Research and Practice* 22.4 (2010), pp. 261–285. DOI: 10.1002/smr.439.

[24] Hong Zhu, Patrick A. V. Hall, and John H. R. May. „Software Unit Test Coverage and Adequacy“. In: *ACM Computing Surveys* 29.4 (1997), pp. 366–427.

Appendix

ME-OA-HGS Minimization Pseudocode

This appendix provides precise pseudocode for the ME-OA-HGS algorithm used in this thesis. The notation follows the set-cover literature [4] and adopts bitset-based operations for efficiency.

Notation

- Input: SRM M from LASSO (stimuli $\times$ systems, structured response objects); target system index j
- Preprocessing: Extract coverage data and mutation information from response objects M_{ij} to construct test–requirement relationships (tests $T = \{t_1, \ldots, t_n\}$ as stimuli; structural requirements $R = \{r_1, \ldots, r_m\}$ as coverage elements; mutants $\mathcal{M} = \{m_1, \ldots, m_p\}$)
- For each test t_i, let $K(t_i) \subseteq \mathcal{M}$ denote the set of mutants killed by t_i
- Optional input: Non-negative weight vector $w \in \mathbb{R}^n$ with $w_i \geq 0$ (execution cost per test, extractable from response objects)
- Parameters: Overlap penalty coefficient $\alpha \in [0, 1]$ (default 0.5); mutation weight $\beta \geq 0$ (default 0.75)
- Output: Reduced SRM M' containing selected stimuli (rows) with original schema preserved

Algorithm

Procedure MEOAHGS(M, w, α, β)
Input: Bitset rows $M[i]$ for tests t_i, mutation kill sets $K(t_i)$
Output: Minimal (heuristic) subset S preserving coverage and mutation kill capability

Phase 1: Initialization

1. Compute requirement cardinalities: $card[j] = |\{i \mid M[i][j] = 1\}|$ for all j

2. Set $S \leftarrow \emptyset$, *covered* $\leftarrow$ all-zero bitset over R

3. Set *coveredMutants* $\leftarrow \emptyset$

Phase 2: Unique Structural Coverage Selection

4. For each requirement r_j with $card[j] = 1$, let i be the unique covering test; add t_i to S

5. Update *covered* $\leftarrow$ *covered* $\vee M[i]$ for each newly added $t_i \in S$

6. Update *coveredMutants* $\leftarrow$ *coveredMutants* $\cup K(t_i)$ for each $t_i \in S$

Phase 3: Mutation-Aware Overlap-Penalized Greedy Selection

7. While *covered* $\neq$ all-ones over target requirements:

7.1 For each remaining test t_i not in S:

$newStruct = M[i] \wedge \neg covered$

$newMutants = K(t_i) \setminus coveredMutants$

$$overlap = \frac{|M[i] \wedge covered|}{\max(1, |M[i]|)}$$

$cost = \max(1, w[i])$ (default 1 if w not provided)

$$\text{Score}(i) = \frac{|newStruct| + \beta \cdot |newMutants|}{cost \cdot (1 + \alpha \cdot overlap)}$$

7.2 Select $i^* = \arg\max_i \text{Score}(i)$

7.3 Set $S \leftarrow S \cup \{t_{i^*}\}$, *covered* $\leftarrow$ *covered* $\vee M[i^*]$

coveredMutants $\leftarrow$ *coveredMutants* $\cup K(t_{i^*})$

Phase 4: Post-processing (redundancy elimination)

8. For each $t_i \in S$ in reverse insertion order:

If $\big(covered \setminus M[i]\big)$ covers all requirements and

$\big(coveredMutants \setminus K(t_i)\big)$ kills all mutants covered by S,

then remove t_i from S and update *covered* and *coveredMutants*

9. Return S

Complexity

Using bitset operations and hash-set representations for mutation kill sets, Phases 1 and 2 run in $O(nm)$, the greedy loop in Phase 3 runs in $O(nm \log m + np \log p)$ where $p = |\mathcal{M}|$ with efficient scanning and priority updates, and post-processing

in $O(k^2(m+p))$ with $k = |S| \ll n$. Overall, the procedure is $O(nm \log m + np \log p)$ time and $O(nm+np)$ space. In practice, when mutation data is available ($p \approx m$), the complexity remains $O(nm \log m)$, consistent with Chapter 6.

A.1. Extended Benchmarks and Reproducibility

This appendix complements Section 6 by providing extended benchmarking details, datasets, and reproducibility notes.

Benchmark Tables

Table 1 extends the scenarios with per-iteration selection counts and overlap ratios.

Table 1.: Extended empirical benchmarks with overlap statistics

extbfScenario	**Orig.**	**Min.**	**Red.**	**Cov.**	**Mean overlap**
Large-scale test	20	9	55.0%	94.0%	0.36
Code coverage	6	4	33.3%	100.0%	0.18
Mutation testing	5	3	40.0%	100.0%	0.22
SRM conversion	5	3	40.0%	100.0%	0.20
Weighted (exec time)	6	3	50.0%	83.3%	0.41

Reproducibility Notes

Experiments are executed on an *Intel Core i7* with 16 GB RAM. Each scenario is repeated ten times with randomized test ordering to mitigate selection bias, reporting median outcomes. Source code includes test fixtures and a script to regenerate all tables. Random seeds and configuration parameters (e.g., α) are documented alongside results.

B.2. SRM Coverage Extraction Specification

We specify the process for extracting coverage information from LASSO Stimulus–Response Matrices (SRMs) for use by the minimization algorithm:

B.2.1. SRM Structure

LASSO SRMs are two-dimensional matrices where:

- Rows (stimuli) represent test sequences, method invocations, or API calls
- Columns (systems) represent executable software units under test
- Cells contain structured response objects M_{ij} with execution results from system j responding to stimulus i

B.2.2. Coverage Extraction Process

For a selected target system (column j):

1. **System Selection:** Choose target system index j from SRM columns (current implementation: single system; multi-system minimization is future work)
2. **Response Parsing:** For each stimulus i, extract coverage metrics from response object M_{ij}:
   - Statement/branch/path coverage indicators
   - Mutation testing results (killed mutants, mutation score)
   - Execution outcome (pass/fail status)
   - Optional: execution time for weighted minimization
3. **Requirement Mapping:** Construct test–requirement relationships:
   - Tests $T = \{t_1, \ldots, t_n\}$ map to stimuli (rows)
   - Requirements $R = \{r_1, \ldots, r_m\}$ map to coverage elements: statement/branch IDs for structural coverage, mutant IDs for mutation testing
   - Coverage indicator: test t_i covers requirement r_k iff response object M_{ij} indicates coverage of element k
4. **Validation:** Filter invalid or inconsistent stimuli (e.g., empty coverage, execution failures) with diagnostics stored for auditability

B.2.3. Output Format

The minimizer produces a reduced SRM M' where:

- Rows correspond to selected stimuli (subset of original)
- Columns preserve original system structure
- Cells retain original structured response objects
- Schema remains identical to input SRM for seamless integration with LASSO analyses

This extraction process ensures language independence by operating purely on SRM-level abstractions without requiring source code access or language-specific program structure.

C.3. Licensing Header Template

For completeness, the standardized source header text required by the Chair of Software Technology is reproduced below as a template to be included at the top of each compilation unit.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, December 1, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, December 1, 2025

Laith Philipp Sandouk